

KV260 LLM 硬件加速测试报告与技术设计文档

最终提交架构、PL 映射、Host-PL 协同与部署流程

AICAS KV260 Acceleration Project

摘要

本文档描述最终 AICAS 提交包中使用的 KV260 PS+PL LLM 加速系统。系统将 llama.cpp 服务端和 Transformer 图控制保留在处理系统侧，同时把选定的 ggml 矩阵运算交给 300 MHz 时序收敛的可编程逻辑加速器执行。PL 核心采用 8×8 int8 dot16 脉动阵列、128-bit 本地存储字、128-bit AXI 数据路径，以及经过 HP/HPC 端口重映射验证的四路 B 侧输入路径。Host 侧运行时通过 ggml FPGA 后端集成该核心，并配合一组经跑分晋升验证的软件优化。最终封装结果得分为 **87.756702485479**，其中准确率为 24/30，prefill 吞吐为 53.380629 tok/s，decode 吞吐为 39.259219 tok/s，能效为 5.613866 tokens/J，TTFT 斜率为 1.441085 ms/char [E01,E02]。本文依次介绍硬件加速架构设计、PL 端算子选型与映射、Host-PL 协同执行方案，以及 KV260 部署和官方测评流程。

目录

1 引言与系统概述	3
1.1 背景与目标	3
1.2 最终交付系统配置	3
1.3 官方结果	3
1.4 技术贡献概述	4
1.5 文档组织	4
2 硬件加速架构设计	4
2.1 图执行中的 Host-DDR-PL 数据流	4
2.2 最终硬件参数	5
3 PL 端算子选型与映射	6
3.1 算子选型	6
3.1.1 将 GEMM 作为加速目标	6
3.1.2 选择性 GEMM 卸载边界	7
3.2 GEMM 到 PL 任务映射	7
3.2.1 任务描述符与 CSR 契约	8
3.2.2 矩阵角色与内部 tiling	9
3.3 PL 数据通路 with 计算微架构	9
3.3.1 A 路径	9
3.3.2 B 路径	9
3.3.3 dot16 PE 阵列	10
3.3.4 D/输出路径	11
3.3.5 峰值计算与代表性实测 case	11
4 Host-PL 协同执行方案	12
4.1 运行时栈与执行循环	12
4.2 已接受 GEMM 节点生命周期	13
4.3 协同软件优化	13

4.3.1	提交粒度与任务边界	13
4.3.2	A/B 加载与布局协同设计	13
4.3.3	预加载、ping-pong 与任务重叠	14
4.3.4	PL 准入与图路径选择	14
4.3.5	输出与回读边界	14
4.3.6	Small-A 合并与多实例提交	14
4.3.7	Decode 图复用与相邻节点协作	14
5	KV260 复赛测试说明	15
5.1	测试目标	15
5.2	硬件与前置条件	15
5.3	SD 卡镜像写入	15
5.4	登录 KV260 并检查交付包	15
5.5	挂载数据集	16
5.6	检查系统并启动 llama-server	16
5.7	准备官方测评目录	17
5.8	运行官方测评	17
5.8.1	精度测试	17
5.8.2	吞吐量测试	17
5.8.3	能耗测试	17
5.8.4	TTFT 测试	17
5.9	合并结果	17
5.10	演示实测结果示例	18
5.11	验收检查点	18

1 引言与系统概述

1.1 背景与目标

本项目面向 KV260 上多模态 LLM 的端到端加速。基线执行路径是由官方 AICAS 跑分脚本驱动的 llama.cpp 服务端。在该路径中，Transformer 控制流、图调度、token 生成以及非 GEMM 算子仍运行在处理系统侧；适合矩阵乘法形式表达的节点则卸载到可编程逻辑。项目目标不是单纯提高某个微基准的 TOPS 数字，而是在准确率、吞吐率、能效和 TTFT 约束下提高官方综合得分。

因此，最终系统被设计为 Host-PL 协同优化的推理栈。Host 负责准备 ggml 张量、量化后的激活缓冲区、高 DDR 地址以及任务描述符；PL 核心执行分块 int8 GEMM；完成后的输出写回 DDR，并重新进入 ggml 图。这个闭环是全文使用的基本执行模型。

1.2 最终交付系统配置

冻结配置由固定 PL 比特流和匹配的 Host 侧运行时包组成。PL 侧是 300 MHz int8 GEMM 引擎，包含 8×8 dot16 PE 阵列、128-bit 本地 SRAM 字、128-bit AXI 数据路径以及四路 B 侧输入路径。Host 侧由 llama.cpp/ggml 将选定的 MUL_MAT 节点下沉到 FPGA 后端，管理 high-DDR 缓冲区和描述符，并将非 GEMM 图逻辑保留在 CPU 上。这个划分就是跑分包使用的最终系统边界。

表 1: 最终交付 KV260 系统

项目	最终交付值
板卡	Xilinx KV260
PL 比特流类型	时序收敛的 300 MHz KV260 核心
硬件交付内容	冻结的 300 MHz clean overlay 与 HWH 包
计算阵列	8×8 PE 阵列，每个 PE 执行 dot16 int8 内积
主要加速算子	ggml MUL_MAT/GEMM 类任务
外部数据路径	128-bit AXI、high-DDR 缓冲区、最终板端路径中的四路 B 侧输入
Host 运行时	冻结的混合 llama.cpp/ggml 运行时包
最终官方 JSON	05_scoring_json/aicas_submission.json
得分摘要 JSON	05_scoring_json/score_summary.json
mock 模式	最终运行环境中关闭

1.3 官方结果

表 2 列出冻结包 JSON 与得分摘要中的官方指标。这些数值是后续文档使用的主要性能基准。

表 2: 最终官方跑分指标

指标	数值	来源字段
准确率	$24/30 = 0.8$	accuracy_ratio_accuracy
Prefill 吞吐率	53.380629 tok/s	throughput_prefill_speed_tps
Decode 吞吐率	39.259219 tok/s	throughput_decode_speed_tps
能效	5.613866 tokens/J	energy_tokens_per_joule
平均功耗	6.446811 W	energy_average_power_w
4096-token 能耗测试耗时	113.175652 s	energy_inference_duration_s
TTFT 斜率	1.441085 ms/char	ttft_slope_ms_per_char
TTFT 截距	2646.037512 ms	ttft_intercept_ms
综合得分	87.7567024854788	score_summary_score

1.4 技术贡献概述

在展开架构、映射、Host 运行时、部署流程和实测证据之前，先概括最终晋升的技术点。

表 3: 高层技术贡献索引

侧别	已晋升的贡献点
硬件	dot16 int8 PE 阵列；128-bit AXI/本地 SRAM 粒度；分块 GEMM 任务引擎；A 侧加载路径清理与连续/驻留 A 支持；B 四路输入；HP/HPC 端口重映射；B 本地 bank 计算供给；乒乓缓冲/预加载调度；输出/写回路径与回读同步；CSR/时序路径清理；clean 构建中移除调试计数器；300 MHz 时序收敛的 KV260 bitstream。
软件/运行时	板端 high-DDR 地址窗口安全检查；直接视图矩阵任务缓冲区；可缓存输出 BO；可缓存异步回读 BO；图像编码缓存；图像预处理快速路径；small-A 合并/多实例矩阵任务提交；fast bicubic two-pass；normalize LUT；并行预处理；权重缓存预热；WQKV/GQA row0-mask 处理；RMS/图融合；regular attention 分组；FFN gate/up 同 A 准备结果复用；最终运行脚本对 mock 关闭、PL 时钟、库路径和地址范围检查的固化。

1.5 文档组织

本文后续部分按系统实现链路组织。第 2 章介绍硬件加速架构设计。第 3 章说明 PL 端算子选型与映射。第 4 章描述 Host-PL 协同执行方案。第 5 章给出 KV260 复赛环境部署、服务启动、官方测评命令、结果合并和验收检查点。

2 硬件加速架构设计

加速器作为 llama.cpp/ggml 执行栈中的节点级计算引擎集成。因此，本章从两个面向系统的视角描述硬件架构：一是用于提交并完成一个加速图节点的 Host-DDR-PL 数据流，二是定义最终 KV260 加速器边界的硬件参数。算子级映射和 PE 阵列语义在第 3 章展开。

2.1 图执行中的 Host-DDR-PL 数据流

图 1 是最终加速器使用的系统级数据流。它按所有权划分为左侧 Host 图/运行时路径、中间 high-DDR BO/AXI/CSR 接口，以及右侧 PL TPU 核心。这里最关键的一点是：FPGA 执行是普通 llama.cpp 图内部的节点级服务，而不是单独的最终回答生成阶段。请求首先进入 llama-server，服务端构建并执行 ggml 图；只有当调度器访问到受支持的 GGML_OP_MUL_MAT 节点时，才进入 FPGA 后端。该节点完成后，生成的张量会回到后续图执行；后续 CPU 节点、后续 FPGA 支持节点以及采样仍会继续运行，最后服务端才输出结果。

End-to-end FPGA-supported MUL_MAT path inside llama.cpp graph execution

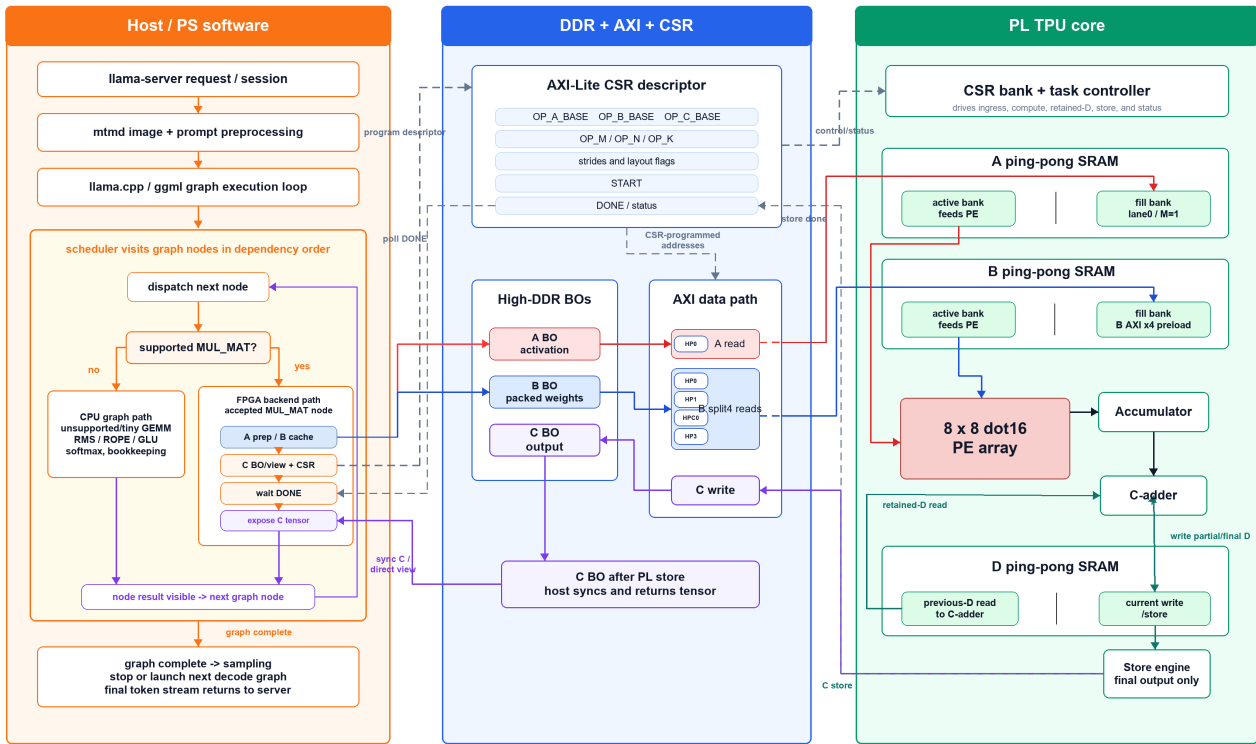


图 1: ggml 调度循环内部一次 FPGA 支持的 MUL_MAT 事务。

Host 侧围绕 ggml 图执行循环组织。调度器逐个访问图节点。不支持或收益不足的节点留在 CPU 路径，包括非 GEMM 算子以及很小或不规则的 GEMM-like 情形。受支持的 MUL_MAT 节点进入 FPGA 后端路径：后端进行形状/layout 判定，准备或复用 B 布局/缓存状态，打包或映射 A 激活数据，准备输出 BO/视图状态，写 CSR 任务描述符，启动 PL 任务，等待完成，在需要时同步 C 缓冲区，并将完成的张量暴露回 ggml。CPU 路径和 FPGA 路径最终在同一个点合流：张量或图侧结果可见，调度器继续下一个图节点。

中间列区分控制、存储与数据传输三个域。CSR 是描述符/控制路径：Host 软件写入 A/B/C 基地址、矩阵尺寸、步长、标志位、START，之后读取 DONE 或 ERROR。High-DDR BO 是存储路径：包括 A 激活 BO、B 打包权重/缓存 BO 以及 C 输出 BO。AXI 是数据传输路径，使用 CSR 编程的地址读取 A、执行 split4 B 读取并写回 C。图中 B 数据路径显式分成四个 AXI 读端口，因为最终 split4 输入通过四个通道填充 B；A 是 lane0-fed，并不是最终设计中的四端口加载。Host 可见的 ggml 张量视图/拷贝决策属于 C 同步后的 Host 返回路径。

PL 侧，CSR 寄存器组与任务控制器构成输入、计算、保留 D 使用、写回和状态的控制平面。A 和 B 都使用左右绘制的本地乒乓 SRAM：一侧喂给 PE 阵列，另一侧接收输入或预加载。A 包含 lane0-fed 的 M=1 驻留/线性合并路径。B 包含四路输入路径以及计算使用的本地 SRAM 供给。D ping-pong SRAM 同样左右分开：一侧接收当前累加结果或将最终输出写回到 C，另一侧可以把前一轮 D/bias 送回 C-adder。中间 D 值可以留在设备上继续累加；当最终输出需要返回 high-DDR C BO 时才触发写回。

这些模块共同定义了加速器的系统契约。Host 图调度选择可接受节点；CSR 描述任务；high-DDR BO 保存张量；AXI 传输张量数据；本地乒乓 SRAM 暂存操作数和结果；dot16 PE 阵列执行分块 GEMM 计算，并在输出张量返回 ggml 前完成写回。

2.2 最终硬件参数

manifest 记录了最终数值参数 [E07]。表 4 不是独立性能表，而是后续文档默认采用的硬件契约。各行应按四组理解：外部 AXI 传输宽度、本地 SRAM 字宽/深度参数、PE 阵列计算拓扑，以及输入/供给行为。这样的分组很重要，因为同一个 128 bit 在多个层级出现但含义不同：AXI beat 表示 DDR 与 PL 之间的数据搬运粒度，而本地 SRAM 字则是 dot16 数据通路消耗的 K16 int8 向量。

表 4: 最终 clean 300 MHz 硬件参数

参数	数值	作用
外部 AXI beat 宽度	128 bit	每条完整 AXI 数据通道的宽度; B 输入使用四条读通道, A 输入不使用四条
本地 SRAM bank 字宽	128 bit	一个本地 A/B bank 字; dot16 模式下是一个 16 个 int8 的向量
PE 阵列规模	8×8	空间脉动阵列
Int8 dot factor	16	每个 PE 的内积宽度
M 分块	最多 32	输出行维度上的内部任务块
N 分块	128	输出列维度上的任务分块
K 分块	128	归约维度上的任务分块
A 本地地址宽度	5	本地 A 存储深度参数
B 本地地址宽度	7	本地 B 存储深度参数
D 本地地址宽度	8	本地输出/D 存储深度参数
A 输入	lane0/HP0 路径	支持连续/驻留 A; 最终 split4 路径中不是四端口 A 加载
B 输入	四条 AXI 读通道	DDR 到 B SRAM 的 split4 加载, 最终板端映射到 HP0/HP1/HPC0/HP3
阵列 B 计算供给	8 向量/beat	$8 \times 128 \text{ bit} = 8 \times 16 \times 8 \text{ bit/beat}$; 8 个 B 列 K16 向量喂入 PE 列
阵列 A 计算供给	最多 8 向量/beat	$8 \times 128 \text{ bit}$ 物理行向量端口; 每个有效行是一个 A K16 向量, M=1 时只有 row0 有效

表 4 也区分了输入带宽和计算供给带宽。B 输入使用四条外部 AXI 读通道填充本地 B SRAM, 但 PE 阵列后续从 SRAM bank 中消费本地 B K16 列向量。A 输入是 lane0-fed 并支持驻留/连续; 这不同于阵列看到的物理行向量供给。这些区别会在第 3 章解释 A 加载、B split4 加载、dot16 PE 语义和 D/输出暂存时继续使用。

3 PL 端算子选型与映射

3.1 算子选型

3.1.1 将 GEMM 作为加速目标

算子选型首先来自 CPU 代价。从最终 fullscore 日志中收集由交付后端卸载的 GEMM 形状, 并投影到 KV260 上实测的 CPU kernel。相同 GEMM 集合如果全部由 CPU 执行, 需要 1,618.550 s; 因此把这部分矩阵工作留在 Cortex-A53 上无法满足官方得分目标 [E20,E22,E23]。

该 CPU 投影绑定的是 GEMM 热点路径, 而不是偶然的图开销。最终后端计时也支持这个边界。在最终 4096-token 时间分账中, GEMM 是最大的命名后端分账桶: activation/quantize 为 37.076 s, GEMM 为 52.598 s, 输出反量化为 19.385 s [E18]。更关键的是, 如果将已卸载 GEMM 计算替换为实测 CPU 投影, 而其他命名后端分账桶保持不变, 则投影后的命名后端范围为

$$79.814 + 1618.550 = 1698.364 \text{ s,}$$

其中被替换的 GEMM 计算占

$$1618.550/1698.364 = 95.3\%$$

[E23]。这说明 GEMM/MUL_MAT 是正确的 PL 端目标: 它不只是出现在图里, 而是主导了可替换计算。

随后, 将同一批记录到的工作量与 PL 卸载执行对比。当前相同 GEMM 任务集合的 PL 等待时间为 133.814 s, 而 CPU 投影为 1,618.550 s。对于精确记录的卸载形状集合, 任务时间比为 12.10x, 其中提示词/图像侧大 M 工作和 decode 中 M=1 主导工作都提供了贡献。

表 5: 最终 fullscore 日志中实际卸载 GEMM 形状的 CPU-to-PL 对比

已记录卸载 GEMM 分组	调用次数	CPU-only 投影	当前 PL 等待	CPU/PL 比值
提示词/图像侧大 M 分组	11,946	919.109 s	25.586 s	35.92x
Decode / M=1 主导分组	1,264,429	699.441 s	108.228 s	6.46x
已记录卸载 GEMM 合计	1,276,375	1,618.550 s	133.814 s	12.10x

因此，交付设计选择 GGML_OP_MUL_MAT/GEMM 作为 PL 执行对象，是因为它同时满足三个条件：纯 CPU 执行过慢，GEMM 主导可替换后端计算，PL 实现能将同一记录到的矩阵任务集合缩短一个数量级以上。表中数据不是猜测的完整纯 CPU 官方跑分，而是基于实测 CPU 微基准和最终 fullscore 服务端日志的同形状替换投影。

3.1.2 选择性 GEMM 卸载边界

选择 GEMM 作为 PL 算子，并不意味着每一个 GEMM-like 图节点都应该送入硬件。交付 PL 内核是分块 int8 GEMM 引擎，不是通用 ggml 图执行器。一个节点只有在后端能把它安全表达为一个矩阵描述符时才可以进入 PL：包括 A/B/C 基地址、stride、逻辑形状 (M, N, K) 和 CSR 控制字段。不支持的 view、不规则 batch 语义、不支持的输出类型，以及低于收益下限的形状都保留在 CPU 上。

表 6: 最终 GEMM 卸载边界

类别	运行时决策	主要边界
PL GEMM	提交一个 CSR 描述的矩阵任务	dtype、layout、batch 关系、output type 和形状受支持； $K \geq 64$ ， $N \geq 64$
CPU GEMM 回退	保留在 CPU ggml 路径	不支持的 view/stride/batch/output，或形状太小/太碎，PL 固定开销不划算
CPU 图工作	保留在 CPU 图路径	ROPE、RMS、softmax、GLU、sampling、attention 记录维护、scheduling 等非 GEMM 逻辑

在最终代码中，该策略由 FPGA 后端支持判定函数实现。后端在声明 PL 支持某个 MUL_MAT 节点之前先进行准入判定，并记录拒绝情形的诊断原因。该判定要求存在受支持的源张量转换路径、交付路径中的 F32 输入/输出类型、可表示的 2D 或显式允许的 row-dense 平面、可整除的 batch 维度，以及 $K \geq 64$ 、 $N \geq 64$ 。ViT attention strided 情形只有在 type/shape/layout 精确匹配时才被接受。最终边界比纯 dense 2D GEMM 更宽，但有意窄于任意 ggml view 或融合 attention 闭包 [E21,E24]。

最终 fullscore 日志在运行时上确认了相同边界，并且没有把调度器支持查询误当成真实 CPU kernel。在提示词/图像侧部分，后端启动了 11,946 个 PL GEMM 实例，观察到的范围是 $M = 64-1024$ 、 $N = 64-3072$ 、 $K = 64-12288$ 。在 decode 主导部分，后端启动了 1,264,429 个 PL GEMM 实例，观察到的范围是 $M = 1-512$ 、 $N = 576-49280$ 、 $K = 576-1536$ [E24]。部分提示词/图像侧支持检查被准入判定拒绝，但这些检查是调度器的能力查询，不是 CPU 执行的回退 GEMM 的计数。详细拒绝日志保存在证据文件中，主要原因是不支持的 dense-plane/view 条件，另有少量 $K < 64$ 或 $N < 64$ 情形 [E24]。

因此，该策略既不是“所有 prompt/ViT 工作都留在 CPU”，也不是“所有 GEMM-like 节点都进入 PL”。可接受节点进入 PL，是因为它们能被描述符安全表达并且有足够收益；被拒绝节点或小节点因正确性或固定开销收益保留在 CPU 上。调用/MAC 分布解释了最终权衡： $M = 1$ 贡献 98.051% 的卸载调用，但只贡献 21.221% 的 MACs，因此 decode 对任务粒度和 B 加载压力很敏感； $M > 1$ 只贡献 1.949% 的调用，却贡献 78.779% 的 MACs，因此大 M 是算术密度优势。交付设计在描述符能安全、有效表示时加速 GEMM。

3.2 GEMM 到 PL 任务映射

3.2.1 任务描述符与 CSR 契约

完成算子选择后，选中的图节点被映射成硬件任务。进入 PL 的外部任务 **不是** 固定的 128-by-128 分块。Host/后端将逻辑矩阵任务写入 CSR 寄存器，包括基地址、形状 M, N, K 和 stride。随后硬件任务控制器在内部遍历逻辑矩阵：使用最多 32 行的 M 块以及 128 大小的 N/K 分块循环，加载 A/B 到本地 SRAM，执行 dot16 PE 阵列步骤，累加部分和，并在需要时写回最终 C 区域。

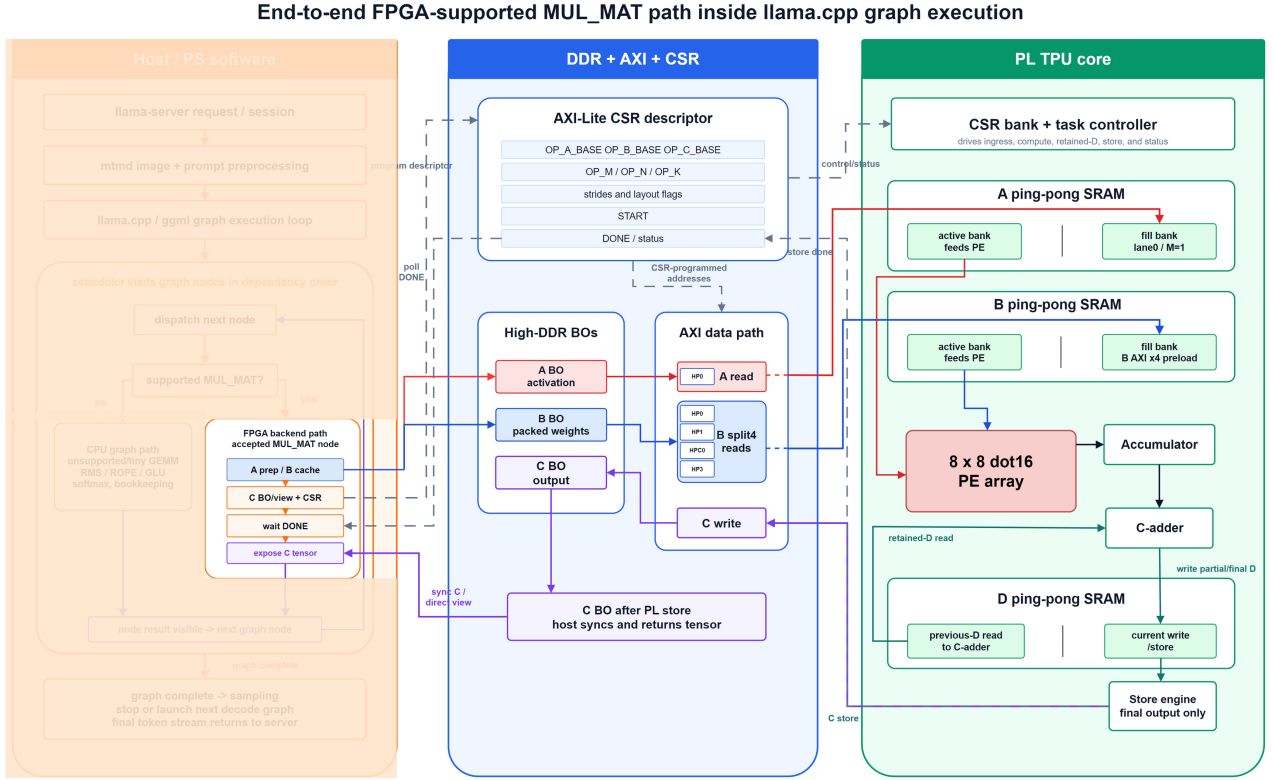


图 2: 一个被接受的 MUL_MAT 节点到 CSR、本地缓冲区、PE 计算和 C 写回的映射。

图 2 展示的是映射契约，而不是一个独立的加速器流水线。Host/后端首先把一个被接受的 MUL_MAT 节点转换为 CSR 描述符：逻辑 M, N, K 、A/B/C 基地址、stride、布局标志以及启动/状态字段。这些 CSR 字段控制 PL 任务控制器，而矩阵数据仍位于 high-DDR BO 中。A 使用激活 BO，B 使用打包权重/缓存 BO，C 使用输出 BO；C 之后会被同步或暴露回 ggml 图。

图中的 AXI 数据路径按操作数角色拆开。A 通过 A 读取路径加载，B 通过四路 B 读取路径提供，C 在任务输出完成后通过 C 写回路径写回。PL 侧，A 和 B 先进入 ping-pong SRAM，再喂给 8×8 dot16 PE 阵列。Accumulator 和 C-adder 在需要跨内部分块累加时，将 PE 部分和与保留 D 数据结合；D ping-pong SRAM 保存中间结果或最终输出状态，写回引擎再将 C 写回 DDR。该路径的结果仍然是图张量，而不是语言模型的答案：C BO 被 Host 同步或创建视图后，ggml 继续执行下一个图节点。

表 7: 一个映射后的 GEMM 任务使用的 CSR/任务字段

CSR 字段组	典型内容	映射作用
A 基地址/步长	OP_A_BASE, A row/tile stride	标识逻辑 $M \times K$ 激活 BO 区域
B 基地址/步长	OP_B_BASE, B K/N strides 或 packed-layout stride	标识逻辑 $K \times N$ 打包权重/缓存 BO 区域
C 基地址/步长	OP_C_BASE, C row/tile stride	标识逻辑 $M \times N$ 输出 BO 区域
形状	OP_M, OP_N, OP_K	后端提交的完整逻辑形状; 不限于 128
内部分块参数	硬件清单中的常量	M 在内部最多按 32 行块; N-tile 和 K-tile 为 128 [E07]; 这些定义循环粒度, 而不是外部算子尺寸
控制/状态	START, DONE, 错误/状态位	启动任务, 控制 Host 等待/同步, 并报告完成或错误

表 7 是该契约的具体形式。Host 写入一个逻辑矩阵描述符, 控制器负责内部循环顺序、本地缓冲区选择、累加和完成状态。Decode 中 M 可以小到 1, 而 N 和 K 仍必须满足后端支持判定和收益下限才能进入 PL。

3.2.2 矩阵角色与内部 tiling

对于 $C = A \times B$, 映射后的逻辑角色是固定的: A 是形状为 $M \times K$ 的激活矩阵, B 是形状为 $K \times N$ 的权重/缓存矩阵, C 是形状为 $M \times N$ 的输出矩阵。最终硬件使用最多 32 行的内部 M 块, 并配合 $N = 128$ 、 $K = 128$ 的分块循环 [E07]; 这些是在被接受的 CSR 描述符提供逻辑形状之后由控制器使用的分块尺寸, 而不是图节点本身的大小。在 prefill 或大 M 任务中, 一个逻辑任务包含足够多计算量来摊薄加载/写回开销; 在 M=1 decode 中, A 很小而 B 仍然很大, 因此 B 加载和任务包络开销会变得明显。

3.3 PL 数据通路 with 计算微架构

GEMM 节点映射为 CSR 描述的任务后, 硬件实现由 DDR、本地 SRAM、PE 阵列和输出路径中的 A、B、D/C 路径定义。这些模块不是算子选择规则本身, 而是让映射任务在 KV260 上可执行且高效的数据通路实现。

3.3.1 A 路径

A 路径承载 $C = A \times B$ 中的激活侧, 因此逻辑形状为 $M \times K$ 。关键是区分外部输入与本地阵列供给。在外部 DDR 侧, 最终 split4 设计没有为 A 使用四个读端口: A 通过 lane0 写入, 而新增的其他读通道分配给 B。最终 tpu_top.v RAM 选择注释记录了这一点 [E15]。

A 进入本地 SRAM 后, 计算侧视图更宽。data_flow_load.v 在 int8-dot 窗口中读取本地 A bank, 并将 int8_dot_systolic_a_wr_data 组织为送入脉动阵列的行向量块 [E15]。也就是说, 外部 A 加载路径可以较窄, 而本地 PE 供给仍然向阵列提供行 K16 向量。这一区分避免把 A 路径误解为四路 B 输入。

最终设计还包括来自早期板端诊断的两个 A 侧清理。第一, 当行步长与行大小匹配时, 连续 A 行保持为线性突发传输, 避免板端测试中出现的碎片化行突发传输行为。第二, 小型 resident-A 情形可以在重复 N-tile 工作期间将很小的激活数据占用保留在本地。这些变化降低了固定 A 侧开销; 这些优化到位后, 高频 M=1 decode 主要受限于 B 供给和任务包络开销, 而不是 A 输入 [E16]。

3.3.2 B 路径

B 是权重/缓存侧矩阵, 逻辑形状为 $K \times N$ 。在 M=1 decode 中, A 行很小但 B 分块仍然很大, 因此 B 是加载主导的操作数。最终 B 路径因此分成外部填充阶段和内部计算供给阶段。

外部阶段通过四条 128-bit AXI 读通道将 B 从 DDR 搬入本地 B SRAM。最终板端映射是 lane0 HP0、lane1 HP1、lane2 HPC0、lane3 HP3。这个映射不是装饰性改动: 阶段标签计数器显示高 PL 时钟下存在 B 侧 RVALID 饥饿与 HP1/HP2 输入竞争, 将其中一条 lane 移到 HPC0 改善了 M=1 板端性能 [E10,E11,E12]。

内部阶段与外部输入不同。计算时, `data_flow_load.v` 读取全部八个本地 B bank, 并将 `int8_dot_systolic_b_wr_data` 组织为八个 128-bit K16 列向量。因此 PE 阵列每个供给 beat 消费 8×16 个 B int8 数值, 尽管 DDR 填充阶段只有四条 AXI 通道 [E15]。B SRAM 使用乒乓缓冲: 一组 bank 喂给 PE 阵列, 另一组 bank 可以为后续内部分块进行填充或预加载。大 M 工作可以利用这种重叠; M=1 decode 往往无法完全隐藏 B 填充成本, 因为它的计算窗口很短。

3.3.3 dot16 PE 阵列

PE 阵列是 A 和 B 本地供给转换成计算量的位置。在 `systolic_array.v` 边界, 两侧物理端口都按如下方式打包:

$$\text{left_all} : 8 \times 128\text{b}, \quad \text{up_all} : 8 \times 128\text{b}.$$

每个 128-bit 分组是包含 16 个有符号 int8 数值的 K16 向量。两个操作数的矩阵角色不同: A 行向量从左侧进入并沿输出列方向水平传递, B 列向量从上方进入并沿 PE 列垂直传递。因此, 一个代表性 PE 会为一个输出位置计算一个部分和:

$$A_{\text{vec}} = [A[m, k], A[m, k+1], \dots, A[m, k+15]], \quad B_{\text{vec}} = [B[k, n], B[k+1, n], \dots, B[k+15, n]],$$

$$\text{dot16}(m, n, k) = \sum_{i=0}^{15} A[m, k+i] \cdot B[k+i, n].$$

即使是 $32 \times 128 \times 128$ 这样的硬件任务分块, 也不能作为一个整体一次性进入 PE 阵列。在 `data_flow_load.v` 内部, 任务会进一步分解为 PE 阵列工作组: `block_m_cnt` 遍历最多 8 行一组的行组, `block_n_cnt` 遍历 8 个输出列一组的列组, K 维则以 16-lane dot 向量送入。对于 $M = 32, N = 128, K = 128$, 这意味着 4 个行组、16 个列组, 以及每个 $C[8 \times 8]$ 输出块的 8 个 K16 累加步骤 [E15]。

对于一个 $C[8 \times 8]$ 输出块, 控制器反复从当前 A 行组和 K 窗口取 $A[8 \times 16]$, 从匹配的 K 窗口和输出列组取 $B[16 \times 8]$, 并把乘积累加到同一个 $C[8 \times 8]$ 输出块中。随着 K 前进, A 中被选中的窗口沿 A 的 K 方向移动, B 中被选中的窗口沿 B 的 K 方向移动; 只有 K 维累加完成后, 控制器才移动到另一个 N 组或 M 组。这就是较大的硬件任务分块与每周期 PE 供给之间的具体桥接关系。

空间上, 8×8 PEs 构成这个 8 行 $\times$ 8 列输出块。在完整的 $M \geq 8$ 供给步骤中, 最多八个 A 行 K16 向量喂给八个 PE 行, 八个 B 列 K16 向量喂给八个 PE 列。在 M=1 decode 中, 只有 row0 有效; 其他物理行槽位对有效计算来说是填充或非活跃状态。时间上, `PE.v` 会寄存符号扩展后的 A/B 通道、DSP 乘积、乘积重放级、level-1/level-2/level-3 部分和以及最终 dot16 和 [E15]。因此, 该阵列既是空间脉动阵列, 也是内部流水的计算结构; 它不是一个没有寄存的 16 乘积组合加法树。

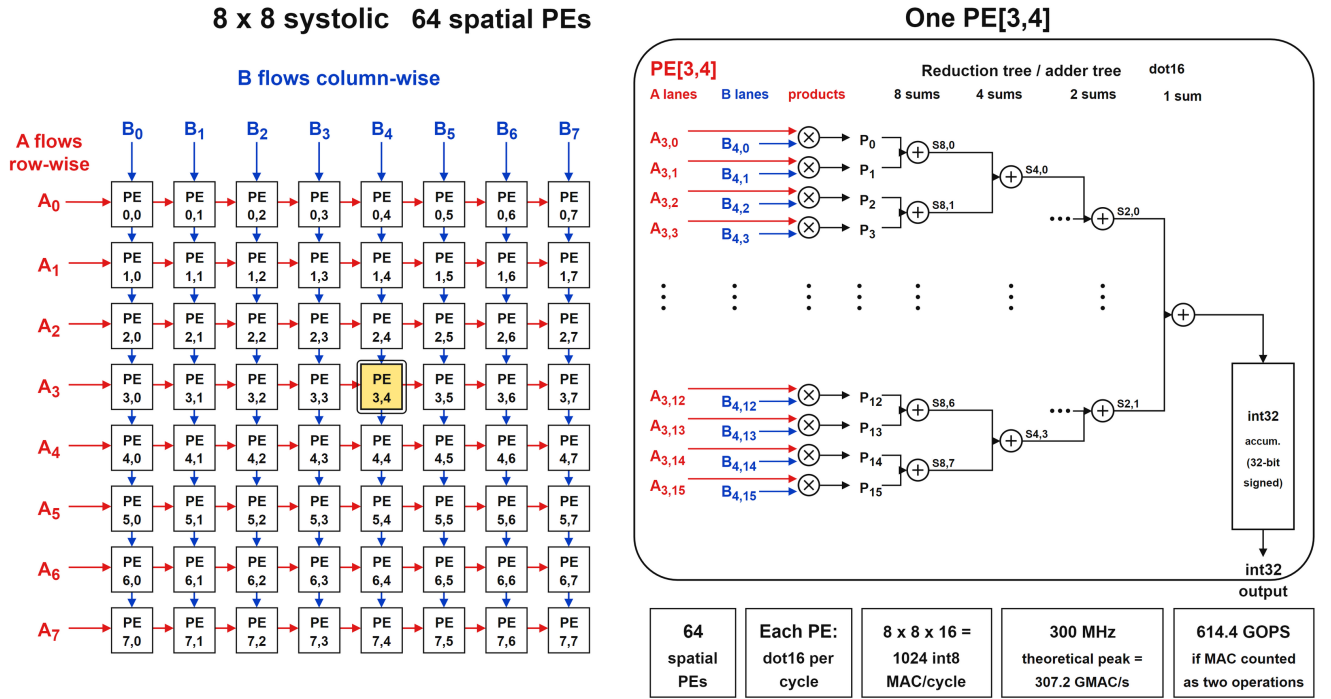


图 3: 文章级 dot16 数据流。图中展示矩阵块上下文、8 x 8 PE 阵列供给方向以及单 PE dot16 运算，避免嵌入仅调试用仿真 trace。

图 3 从矩阵块、阵列供给和单 PE 三个视角可视化同一边界。左侧展示正在计算的矩阵块，中间展示 A 行向量和 B 列向量如何进入 8 x 8 阵列，右侧放大一个 PE 的 dot16 运算。关键理解是：A 是沿 PE 列复用的 K16 激活向量，B 为不同输出列提供 K16 向量；两个向量都包含 16 个不同的 int8 K 通道数值。

3.3.4 D/输出路径

输出侧不是从 PE 阵列到 DDR 的一根直连线。PE 部分和首先进入 accumulator/C-adder 路径，根据任务需求与正在累加的部分结果或前一轮 D 值结合。D ping-pong SRAM 是本地临时结果存储：一侧可以保存或喂出中间 D/C 值，另一侧接收当前结果或准备最终写回。只有逻辑输出分块完成后，写回引擎才将最终 C 区域写回 high-DDR C BO。

该输出路径完成 GEMM 任务，对正确性很关键；但最终 M=1 诊断显示，它不是 decode 的主要瓶颈。更大的压力来自 B 读取供给和固定 Host/任务包络开销；D/C 暂存主要解释部分和如何留在设备上直到输出完成。

3.3.5 峰值计算与代表性实测 case

PE 阵列内部峰值为：

$$\begin{aligned} \text{MAC/cycle} &= 8 \times 8 \times 16 = 1024, \\ \text{GMAC/s at 300 MHz} &= 1024 \times 300\text{M} = 307.2, \\ \text{GOPS if 1 MAC} &= 2 \text{ ops} = 614.4. \end{aligned}$$

这只是阵列内部数值。它不包含 A/B 加载、D/C 暂存、写回、DDR 行为、CSR/任务包络开销和 Host/后端工作。代表性 RTL 与板端测量展示了该峰值与端到端加速器任务行为之间的差距。

表 8: 最终硬件线的代表性 RTL 仿真与板端测量

证据类型	形状 $M \times N \times K$	时间来源	有效 GOPS
RTL 仿真	1 x 128 x 128	499 cycles	19.700
RTL 仿真	32 x 128 x 128	2251 cycles	139.748
RTL 仿真	1024 x 768 x 768	760263 cycles = 2.534 ms	476.911
板端微基准	1024 x 768 x 768	2.986 ms/次调用	404.497
板端微基准	1 x 2048 x 2048	397.7 us @300MHz	21.093
板端微基准	1 x 1024 x 1024	118.1 us @300MHz	17.757

表 8 与解释文本分开，以保持证据可读。RTL 行验证了 $M=1$ 和 $M>1$ 形状の数値完成与顺序；它们是功能和周期级检查，并不能替代板端 DDR 行为。板端行解释了为什么同一个阵列内部峰值不会在所有工作负载上转化为相同有效 GOPS。大 M 任务让阵列保持繁忙的时间足够长，可以摊薄加载和任务开销；而 $M=1$ decode 即使有四路 B 输入与乒乓缓冲，也会暴露 B 加载、同步和描述符开销 [E10,E11]。

4 Host-PL 协同执行方案

本章说明最终运行时如何连接软件图执行与 PL GEMM 任务。核心要点很简单：加速器处理选定的 MUL_MAT 节点，而 llama.cpp/ggml 仍负责周围图逻辑、非 GEMM 算子、采样和响应生成。

4.1 运行时栈与执行循环

图 4 应被理解为一 次反复穿过软件栈的执行循环，而不是 PL 直接生成最终答案的单向路径。请求首先进入服务端，多模态前端准备图像和提示词张量，llama/ggml 构建并执行模型图。大多数图工作仍是普通 CPU 执行。只有当调度器到达硬件支持的 MUL_MAT 节点时，图才调用 FPGA 后端。



图 4: 分层运行时循环：选定 GEMM 节点下沉到 PL，完成后的张量结果返回 ggml，然后图执行继续。

在一个被接受的后端调用内部，张量元数据被转换为 BO 视图、物理地址和 CSR 任务描述符。PL 执行该分块 GEMM 任务并写结果缓冲区。完成后，后端将输出张量暴露给 ggml，控制流返回图。CPU 随后继续执行 RMS、ROPE、GLU、softmax、sampling、token 记录维护或下一个图节点等后续消费者节点。在生成阶段，每当调度器到达另一个受支持矩阵节点，这种节点级模式就会重复。

该图分离的是责任边界，而不是暗示每层都执行加速。服务端管理请求和会话生命周期。多模态前端准备图像/prompt 输入。llama/ggml 核心管理图顺序和 CPU 算子。FPGA 后端负责被接受的 MUL_MAT 的决策与准备。运行时和 RTL 通过第 3.2 节描述的 A/B/D 路径与 PE 阵列执行真实 CSR 驱动的分块 GEMM 任务。

表 9: 按层划分的运行时源码职责

层级	源码区域	主要职责
服务端	tools/server	请求、会话、生成循环
多模态前端	tools/mtmd	图像和 prompt 预处理
llama/ggml 核心	src, ggml 图	图顺序与 CPU 算子
FPGA 后端	ggml-fpga	硬件准入与任务启动
运行时 / RTL	C 运行时与 RTL 工程	CSR、AXI 数据搬运、分块 GEMM

PL 路径不是端到端推理。它加速准入 GEMM 节点；CPU 负责剩余图逻辑与生成循环。这个划分是第 4.2 节节点级生命周期与第 4.3 节协同软件优化的基础。

4.2 已接受 GEMM 节点生命周期

一次加速操作从调度器到达候选 MUL_MAT 节点开始。FPGA 后端先检查 dtype、shape、layout、view、batch 语义和输出兼容性。如果节点被接受，后端准备 BO/视图状态，写 CSR 任务描述符，启动 PL 任务，等待完成，并把结果张量暴露回 ggml。

这个生命周期是有意保持窄边界的。只有已接受 GEMM 任务的计算量和数据搬运被委托给 PL。CPU 仍执行周围的 ROPE、RMS、GLU、softmax、sampling、attention 记录维护、图遍历和后续张量消费者节点。因此，端到端得分同时取决于 PL 吞吐率和每个已接受任务周围的 Host 侧固定开销。

4.3 协同软件优化

交付系统不是纯 PL 加速器。它的性能来自软件图、张量布局、Host 缓冲区和任务提交边界与硬件 GEMM 引擎的匹配。下面的优化按协同设计角色分组：它们要么让更大、更有价值的工作进入 PL，要么降低 PL 任务周围 A/B/D 准备成本，要么移除相邻 PL 任务之间重复的 CPU 工作。

4.3.1 提交粒度与任务边界

早期运行时路径将硬件分块循环过多暴露给软件。当时 $N/K = 64$ 是软件可见的分块/提交粒度：Host/后端仍需要围绕一个已接受 GEMM 节点的许多小片段支付 CSR、BO、调度器和同步成本。第一步实用优化是将软件可见粒度扩大到 $N/K = 128$ ，减少片段数，同时通过零填充在分块层内处理尾部。后续步骤改变了分块的所有权。通过 OP 描述符/矩阵任务路径，Host 后端将一个已接受 GEMM 节点作为逻辑矩阵任务提交，携带基地址、shape、stride 和模式字段；PL 侧控制器随后遍历内部 $M/N/K$ 分块序列。因此， $64 \rightarrow 128$ 是软件可见分块尺寸的演进，而描述符路径将重复分块循环移到了 PL 内部。

4.3.2 A/B 加载与布局协同设计

对于 decode 主导 $M = 1$ ，B 侧流量是主要压力，因为权重侧矩阵在单 token 内几乎没有复用。因此，硬件移动到四路 B 输入路径；板端测量显示早期 HP/HPC 分配存在 B 侧读饥饿后，又调整了板端映射。软件必须匹配这条硬件路径：B 被重排成本地 B bank 期望的 bank 友好的 B16 布局，让四条 AXI 通道喂入有效数据，而不是在运行时中修复不友好的内存布局。

A 侧也按同样思路对齐。软件准备描述符兼容的连续 int8 A 分块；后端通过融合/NEON 行量化以及 dynamic-src0 准备结果复用降低重复 A 准备，当 CPU 生产者和 FPGA 消费者共享同一个分块窗口时可以复用表示。重点不是一个独立 CPU 技巧，而是把 A 呈现为让 PL 任务用更少短突发传输和重复准备就能启动的形式。

4.3.3 预加载、ping-pong 与任务重叠

预加载路径也经历了阶段演进。最早可工作的路径没有预加载，因此分块级加载、计算和写回基本串行：PE 阵列不能在下一分块所需数据加载完成前开始有用计算。之后的实现加入了软件控制的预加载，允许下一次加载阶段与当前计算阶段重叠，移除了大量空闲间隔，但需要 Host 在正确时刻准备下一数据。最终任务模型中，预加载由硬件管理的内部分块循环控制：软件编程描述符和合法缓冲区状态，PL 控制器推进分块序列，并使用乒乓缓冲区，使一侧喂 PE 阵列，另一侧接收下一分块数据。运行时仍保护完成状态与链边界语义，但稳态预加载时序不再是一个单独的软件操作。

4.3.4 PL 准入与图路径选择

后端不会把每个 GEMM-like 操作都发送到 PL。它接受 dtype、layout、shape、batch 语义和描述符契约兼容的节点；对于很小或不兼容的情形，则在固定启动开销占主导时留在 CPU。在这个规则内，多个优化增加了有价值的 PL 工作。Direct ViT PL-OP 支持让 $M = 1024, N = 1024, K = 64$ 等大型 ViT attention 矩阵乘进入 PL，而不是继续 CPU 回退。其他布局处理则让选定 dense 或 contiguous-plane 情形可被接受，而不是被当作通用不支持视图。

Attention 模式也是图路由决策。当前 OCR/VQA 路线使用 regular attention 替代 flash attention，因为它让更多长 attention 工作通过现有加速路径，并避免 flash attention 中观察到的长输出崩溃。后续 GQA grouping 和 row0-mask specialization 进一步减少了 regular attention 路径周围重复的 CPU 工作。

4.3.5 输出与回读边界

加速器任务只有在 CPU 图能看到结果张量后才算完成。因此，直接视图矩阵任务缓冲区、可缓存输出 BO 和异步回读都是 PL 边界的一部分。直接视图/可缓存输出让 CPU 以更低可见性/拷贝开销消费 PL 写入结果；异步回读则将剩余传输与后续 CPU 工作重叠。同一路径测量显示，受影响 decode 分段墙钟时间约降低 14.5%，显式回读拷贝成本约降低 85%。这是 Host-PL 优化，而不仅是 CPU 微内核改动，因为它改变了 PL 写入输出如何成为有效 ggml 张量。

4.3.6 Small-A 合并与多实例提交

另一个边界开销来自小 A 侧实例。核心优化是合并：当多个兼容实例属于同一个矩阵任务族时，后端将它们合并为更粗粒度的多实例提交，而不是分别启动每个小 A 片段。原始图语义保持不变，但 CSR 编程、BO 处理、同步和 A 搬运被摊销到合并后的任务包络开销上。这就是交付路径中的 small-A 到 larger-A 的合并。

4.3.7 Decode 图复用与相邻节点协作

若干最终优化面向重复 decode 图结构。WQKV 持久化/融合投影降低稳定 decode 投影周围重复的调度和准备成本。RMS_NORM 使用 NEON 路径，并将 RMS 输出与其直接 multiply 消费者融合，移除一次重复图遍历。FFN gate/up 对被处理为同 A 双 GEMM macro-op：gate 和 up 仍保持两个合法原始 GEMM 形状，但在别名和依赖检查通过时，后端在 sibling 提交之间复用 prepared A。这保留了合法 GEMM 形状，同时捕获关键的同 A 复用机会。

总体来看，这些不是孤立参数调整。主要软件贡献是让图边界、张量表示和任务生命周期匹配真实 PL 加速器：更大的逻辑提交、硬件管理的预加载、B/A 布局匹配、选择性 PL 准入、更低成本的输出可见性、small-A 合并和 decode sibling 复用。

5 KV260 复赛测试说明

5.1 测试目标

本章说明 KV260 复赛交付包的复现测试流程，用于验证以下内容：

- 完整 Linux 镜像可写入 SD 卡并启动 KV260。
- 使用 xilinx / xilinx 可登录正常系统。
- 交付包目录存在，脚本可完成 bitstream 加载与运行环境检查。
- llama-server 可在本地端口启动并保持运行。
- 官方测评脚本可输出精度、吞吐量、能耗、TTFT 结果，并合并为提交 JSON。

5.2 硬件与前置条件

项目	要求
板卡	Xilinx KV260，电源和网线连接正常。
存储卡	32 GB 以上 SD 或 microSD 存储卡，建议使用 64 GB 及以上容量，避免镜像写入空间不足。
镜像文件	kv260_mmcblk1_resized32g_initramfs_20260612_1008.img 或最终提交包中的同版本镜像。
主机工具	Linux/WSL 主机或其他可执行 dd 写盘工具的环境。
网络	KV260 与数据集 NFS 服务器在同一可达网络内。
登录账号	用户名：xilinx；密码：xilinx。
交付包路径	/home/xilinx/kv260_llama_server_minimal。
数据集挂载点	示例挂载点为 /mnt/share，实际测试以现场数据集目录为准。

5.3 SD 卡镜像写入

在主机上插入 32 GB 以上存储卡，确认实际设备名后写入镜像。以下命令中的 /dev/sdX 必须替换为真实 SD 卡设备，不能写成系统盘。

```
sudo dd if=kv260_mmcblk1_resized32g_initramfs_20260612_1008.img \  
of=/dev/sdX \  
bs=4M \  
status=progress \  
conv=fsync \  
sync
```

写入完成后，将 SD 卡插入 KV260 并上电启动。若演示视频需要展示完整链路，可保留 dd 命令开始、进度和完成输出。

5.4 登录 KV260 并检查交付包

KV260 正常启动后，通过 SSH 登录：

```
ssh xilinx@<KV260_IP>
```

登录信息：

```
username: xilinx  
password: xilinx
```

进入交付包目录并检查文件：

```
cd /home/xilinx/kv260_llama_server_minimal
ls
```

目录中应包含：

```
README.md SHA256SUMS bitstream harness loader models python_env runtime scripts
```

5.5 挂载数据集

以下 NFS 命令仅作为挂载方式参考。实际测试时，请将服务器地址、导出路径和本地挂载点替换为现场可访问的真实数据集目录，并确认该目录内包含组委会要求的图片数据集和官方 JSON 文件。

```
DATA_MNT=/mnt/share
sudo mkdir -p "$DATA_MNT"
sudo mount -t nfs4 -o vers=4.0 <NFS_SERVER>:<DATASET_EXPORT> "$DATA_MNT"
findmnt "$DATA_MNT"
ls "$DATA_MNT"
```

后续精度测试命令中的 `-i` 参数应填写真实图片目录，`-d` 参数应填写组委会提供的官方 JSON 文件路径。若数据集已经通过其他方式放在本地，也可以不使用 NFS，直接填写实际本地目录。

5.6 检查系统并启动 llama-server

打开终端 1，先执行系统检查脚本：

```
cd /home/xilinx/kv260_llama_server_minimal
./scripts/check_system.sh
```

该脚本会依次完成：

- 加载 bitstream。
- 配置 XRT、DDR、PL 时钟。
- 检查 XDMA、内存窗口和 FCLK 状态。

检查通过时应看到：

```
PREPARE_OK
```

确认 PREPARE_OK 后，再启动 server：

```
cd /home/xilinx/kv260_llama_server_minimal
./scripts/start_server.sh
```

启动通过时应看到：

```
SERVER_OK
url=http://127.0.0.1:8080
```

服务地址：

```
http://127.0.0.1:8080/v1
```

终端 1 保持打开，server 前台运行。停止服务时执行：

```
cd /home/xilinx/kv260_llama_server_minimal
./scripts/stop_server.sh
```

5.7 准备官方测评目录

打开终端 2，执行：

```
cd /home/xilinx/kv260_llama_server_minimal
source python_env/.venv/bin/activate
RUN_DIR=-/kv260_eval_runs/run_$(date +%Y%m%d_%H%M%S)
mkdir -p "$RUN_DIR"
cd "$RUN_DIR"
```

后续测评输出文件均保存在该运行目录中。

5.8 运行官方测评

5.8.1 精度测试

```
DATA_DIR=/path/to/actual/image_dataset
OFFICIAL_JSON=/path/to/official_ocrbench.json
python /home/xilinx/kv260_llama_server_minimal/harness/acc_eval.py \
-i "$DATA_DIR" \
-d "$OFFICIAL_JSON" \
-o acc_eval_results.json
```

输出文件：acc_eval_results.json。

5.8.2 吞吐量测试

```
python /home/xilinx/kv260_llama_server_minimal/harness/throughput_eval.py \
-i /home/xilinx/kv260_llama_server_minimal/harness/test2.jpg \
-o throughput_metrics.json
```

输出文件：throughput_metrics.json。

5.8.3 能耗测试

```
python /home/xilinx/kv260_llama_server_minimal/harness/energy_eval.py \
-i /home/xilinx/kv260_llama_server_minimal/harness/test2.jpg \
-o energy_metrics.json \
--sample_hz 100 \
--power_path /sys/class/hwmon/hwmon2/power1_input \
--power_unit uW
```

输出文件：energy_metrics.json。如果板卡上的 hwmon 编号不同，应以实际 power1_input 路径为准。

5.8.4 TTFT 测试

```
python /home/xilinx/kv260_llama_server_minimal/harness/ttft_eval_multiprompt.py \
-c /home/xilinx/kv260_llama_server_minimal/harness/ttft_config.json \
-o ttft_eval_results.json
```

输出文件：ttft_eval_results.json。

5.9 合并结果

四项测试完成后，在同一运行目录执行：

```
python /home/xilinx/kv260_llama_server_minimal/harness/merge_results.py \
--acc acc_eval_results.json \
--throughput throughput_metrics.json \
--energy energy_metrics.json \
--ttft ttft_eval_results.json \
-o aicas_submission.json
```

最终输出文件：

```
aicas_submission.json
```

建议记录各 JSON 文件的 SHA256，便于提交前复核：

```
sha256sum acc_eval_results.json throughput_metrics.json energy_metrics.json ttft_eval_results.json aicas_submission.json
```

5.10 演示实测结果示例

以下为一次复现演示中的结果摘要，正式提交以实际运行官方脚本生成的 JSON 为准。

项目	示例结果
准确率	0.8000, 24 / 30
吞吐量	Prefill 53.55 tokens/s, Decode 39.25 tokens/s
能效	5.5912 tokens/J, 平均功耗 6.45 W
TTFT	slope 1.4685 ms/char, intercept 2600.84 ms
综合得分	87.72

5.11 验收检查点

测试完成后，应逐项确认：

- 使用 32 GB 以上存储卡写入镜像后，KV260 可正常启动。
- 可通过用户名 xilinx、密码 xilinx 登录系统。
- /home/xilinx/kv260_llama_server_minimal 目录存在且内容完整。
- 实际数据集目录可访问，精度测试命令中的 -i 和 -d 已指向真实路径。
- ./scripts/check_system.sh 输出 PREPARE_OK。
- ./scripts/start_server.sh 输出 SERVER_OK。
- llama-server 在 http://127.0.0.1:8080/v1 可用。
- 官方测评脚本生成精度、吞吐量、能耗和 TTFT 结果文件。
- merge_results.py 生成 aicas_submission.json。
- 结果 JSON 与提交文件已记录 SHA256 校验值。